

LAPORAN PRAKTIKUM
INTERNET OF THINGS II

Modul 4: Komunikasi Asinkron dan WebSockets



Disusun Oleh:

Shavina Naura Putri

25782055

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, S.Kom

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

LATIHAN 4.6

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP HTML* yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawab:

Alasan Kehematan Bandwidth: Teknologi **WebSockets** menggunakan koneksi persisten satu kali buka (TCP/WebSocket handshake) yang membuat terowongan komunikasi dua arah secara *real-time*. Data yang dikirimkan berukuran sangat kecil (hanya payload JSON murni seperti {"led": "1", "suhu": "30"}) tanpa harus mengirim ulang header HTTP yang besar setiap kali ada pembaruan. Sebaliknya, metode **Meta Refresh HTTP HTML** (seperti pada Modul 3) memaksa browser untuk mendownload ulang seluruh halaman web beserta aset dan header lengkapnya setiap beberapa detik secara terus-menerus, sehingga boros penggunaan data (*bandwidth*).

Alasan Layar Tidak Berkedip: Karena WebSockets hanya memperbarui variabel data tertentu di dalam memori halaman (menggunakan manipulasi DOM JavaScript seperti `document.getElementById()`), tampilan visual di layar browser berubah secara instan tanpa perlu memuat ulang (*reload*) halaman. Pada Meta Refresh, browser wajib me-refresh halaman secara total dari awal, yang menyebabkan layar berkedip setiap kali pembaruan terjadi.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Jawab:

Jika Anda nekat memanggil fungsi `delay(3000)` di dalam siklus `void loop()`, mikrokontroler ESP8266 akan **berhenti total (freeze/blokir)** selama 3 detik penuh pada setiap perulangan. Bahaya sistematis yang akan terjadi meliputi:

1. **Koneksi WebSocket Terputus / Timeout:** Server asinkron membutuhkan siklus CPU yang terus-menerus siaga untuk memelihara koneksi jaringan dan menangani klien. Pemblokiran 3 detik akan membuat server gagal merespons *ping/pong* WebSocket, berujung pada terputusnya koneksi klien (*disconnect*).
2. **Respon Tombol Fisik Lambat / Macet:** Penekanan tombol fisik (*push button*) memerlukan pembacaan status pin yang cepat dan presisi. Jika CPU sedang tidur karena `delay()`, sentuhan tombol pada saat itu tidak akan terbaca sama sekali oleh program.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

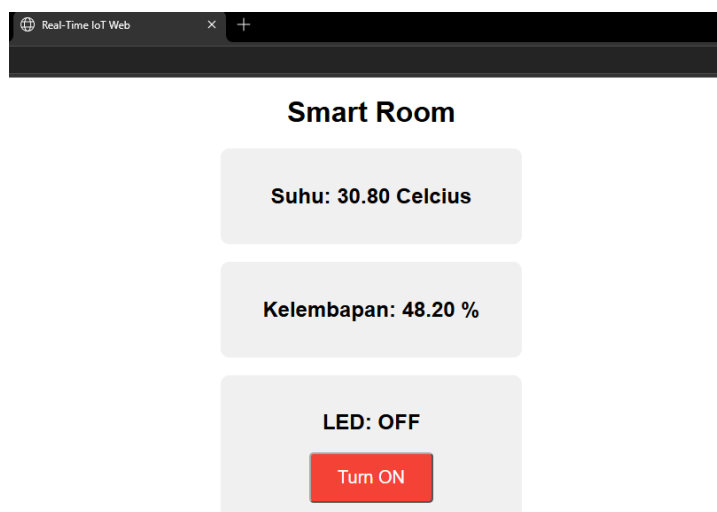
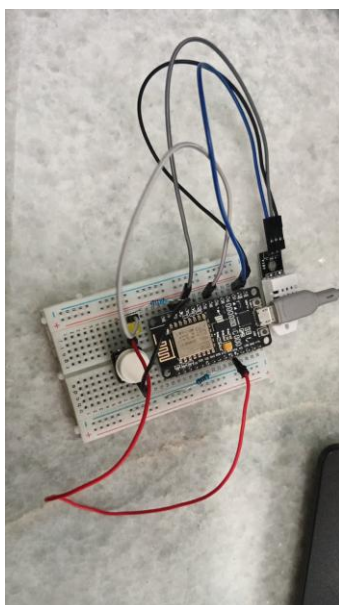
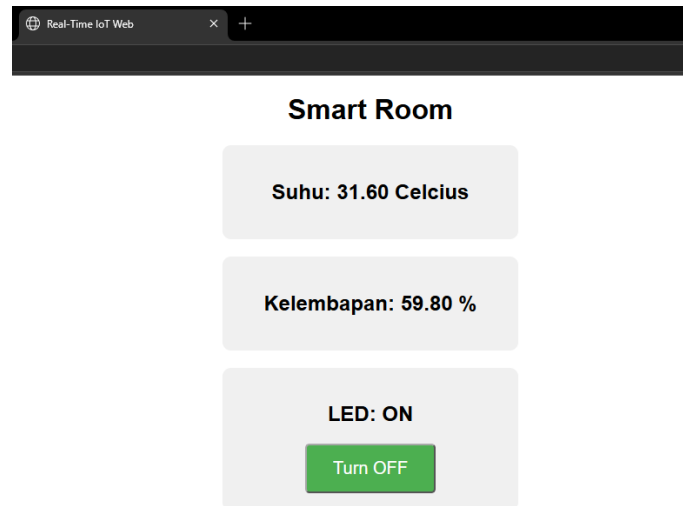
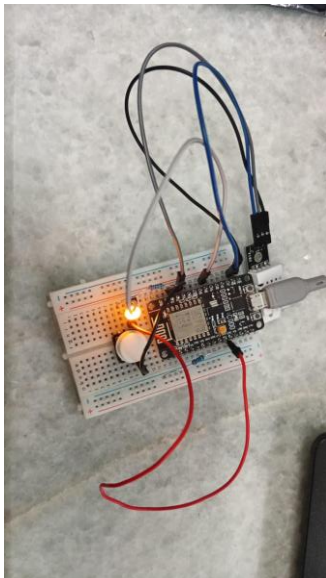
Jawab:

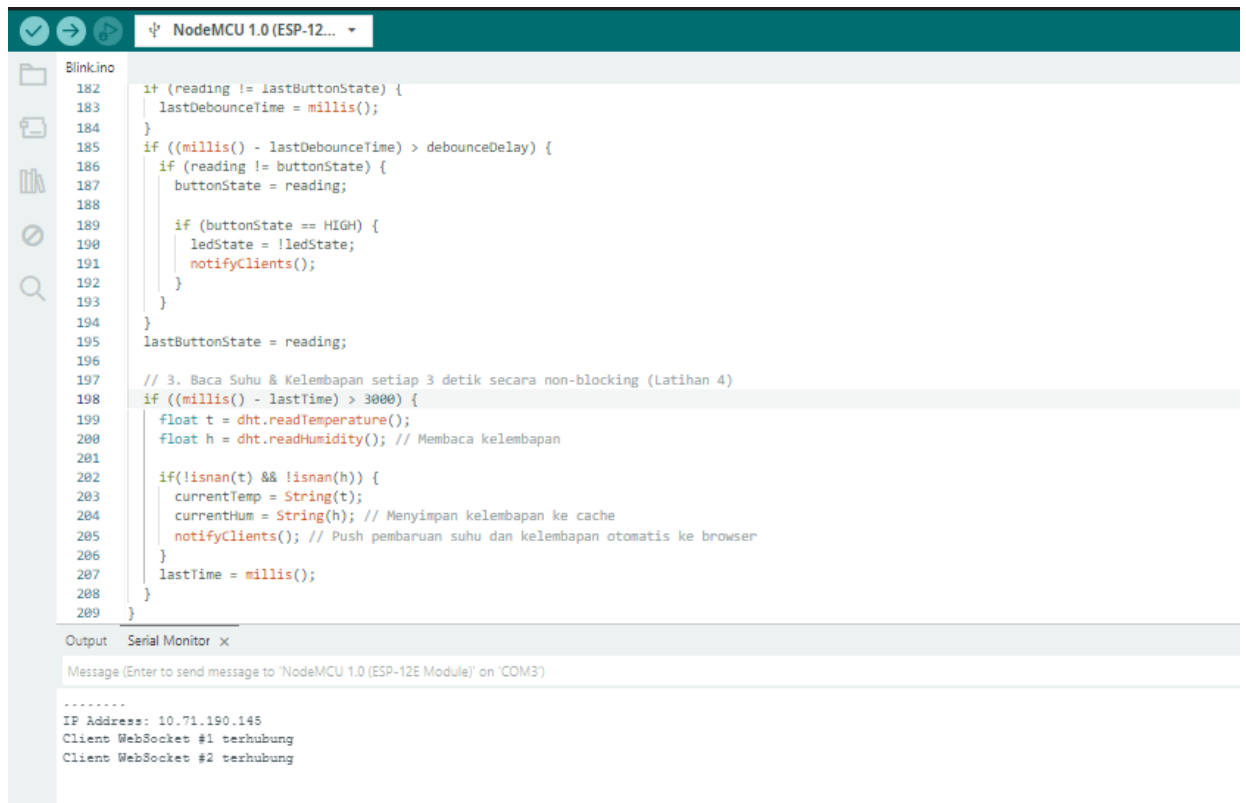
Berdasarkan blok kode JavaScript pada bagian *onMessage*, nama ID elemen HTML (`<span id="...">`) yang menjadi target injeksi/pengubahan teks adalah:

1. `tempValue`: Target untuk menyuntikkan data teks angka suhu ruangan dari sensor.
2. `toggleStatus` (atau `ledStatus` pada script utama): Target untuk mengubah status teks indikator LED (antara "ON" atau "OFF").
3. `toggleBtn`: Target untuk mengubah teks tombol interaktif (antara "Turn ON" atau "Turn OFF").

- Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
 - Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--");. Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer non-blocking millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke browser (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
 - Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.
 Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

Jawab:





```
182 if (reading != lastButtonState) {
183     lastDebounceTime = millis();
184 }
185 if ((millis() - lastDebounceTime) > debounceDelay) {
186     if (reading != buttonState) {
187         buttonState = reading;
188
189         if (buttonState == HIGH) {
190             ledState = !ledState;
191             notifyClients();
192         }
193     }
194 }
195 lastButtonState = reading;
196
197 // 3. Baca Suhu & Kelembapan setiap 3 detik secara non-blocking (Latihan 4)
198 if ((millis() - lastTime) > 3000) {
199     float t = dht.readTemperature();
200     float h = dht.readHumidity(); // Membaca kelembapan
201
202     if(!isnan(t) && !isnan(h)) {
203         currentTemp = String(t);
204         currentHum = String(h); // Menyimpan kelembapan ke cache
205         notifyClients(); // Push pembaruan suhu dan kelembapan otomatis ke browser
206     }
207     lastTime = millis();
208 }
209 }
```

Output: Serial Monitor x

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

IP Address: 10.71.190.145
Client WebSocket #1 terhubung
Client WebSocket #2 terhubung

1. Sisi Back-End (C++)

Bagian kode yang ditambahkan/diubah untuk membaca kelembapan dan mengirimkannya ke format JSON:

C++

// Tambahkan variabel penampung kelembapan

String currentHum = "--";

// Perbarui fungsi notifyClients() agar menyisipkan data "hum"

```
void notifyClients() {
    String jsonString = "{\"led\":\\"" + String(ledState ? 1 : 0) + "\"", ";
    jsonString += "\"suhu\":\\"" + currentTemp + "\"", ";
    jsonString += "\"hum\":\\"" + currentHum + "\"}";
    ws.textAll(jsonString);
}
```

// Di dalam fungsi loop(), pada timer non-blocking (millis)

```
if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity(); // Membaca kelembapan
```

```
    if(!isnan(t) && !isnan(h)) {
        currentTemp = String(t);
        currentHum = String(h); // Simpan kelembapan ke variabel
        notifyClients(); // Kirim data ke browser otomatis
    }
    lastTime = millis();
}
```

2. Sisi Front-End (HTML & JavaScript)

Bagian kode untuk menampilkan elemen kartu kelembapan di HTML dan fungsi JavaScript untuk menangkap datanya:

- **Struktur HTML (di dalam <body>):**

HTML

```
<div class="card">
  <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>
```

- **Modifikasi Fungsi JavaScript (onMessage):**

JavaScript

```
function onMessage(event) {
  var dataObj = JSON.parse(event.data);

  // Menyuntikkan teks angka Suhu
  if(dataObj.suhu !== undefined) {
    document.getElementById('tempValue').innerHTML = dataObj.suhu;
  }

  // Menyuntikkan teks angka Kelembapan (Latihan 4)
  if(dataObj.hum !== undefined) {
    document.getElementById('humValue').innerHTML = dataObj.hum;
  }

  // Merombak UI Tombol & Teks LED
  if(dataObj.led !== undefined) {
    var btn = document.getElementById('toggleBtn');
    var status = document.getElementById('ledStatus');
    if(dataObj.led === "1"){
      status.innerHTML = "ON";
      btn.innerHTML = "Turn OFF";
      btn.className = "btn-on";
    } else {
      status.innerHTML = "OFF";
      btn.innerHTML = "Turn ON";
      btn.className = "btn-off";
    }
  }
}
```